Progressive Optimization of Mixture-of-Experts Inference on GPUs

15-418/15-618 Final Project Proposal
Team: [Member 1] [Member 2]
Project URL: [Project webpage URL]

Summary

We will investigate GPU work scheduling for the expert computation in mixture-of-experts (MoE) inference. Starting from a simple per-expert baseline, we plan to progressively optimize execution and evaluate how workload imbalance and problem size affect performance.

Background

An MoE layer routes each input token to a small subset of expert networks. Each expert processes its assigned tokens using its own parameters, and the resulting outputs are combined for each token. This activates only part of the model for a given input, but produces expert workloads that can vary substantially in size.

Our project focuses on the computation after routing, particularly the expert matrix multiplications, rather than implementing a complete serving system. Tokens assigned to an expert can be processed in parallel, and work across experts is also independent once routing is known. We will use controlled routing assignments and inputs to study how best to map this irregular work onto the GPU.

The Challenge

Uneven routing can leave some experts with many tokens and others with few or none. Launching work separately for each expert can lead to small GPU workloads and repeated launch overhead, while a single large expert may become a straggler. The tradeoff depends on the number of tokens and experts, the distribution of assignments, and the matrix dimensions.

Efficient execution also requires balancing scheduling overhead against useful computation. Expert weights can be reused across tokens, but small or irregular batches may limit reuse and GPU utilization; gathering and scattering routed tokens can add memory traffic. Thread-block scheduling, tile size, and the available parallelism can affect occupancy and load balance. Routing must be available before expert computation, and results must be mapped back to their original tokens. We hope to learn when additional scheduling machinery pays for itself and which bottlenecks dominate under different workloads.

Resources

We plan to develop a small, purpose-built benchmark and GPU implementation, starting from a straightforward baseline and using CUDA and available GPU libraries where appropriate. We will use synthetic inputs and routing assignments so that workload size and imbalance can be varied systematically. We plan to rent a single NVIDIA GPU instance for development and evaluation, subject to availability and cost. We will also investigate course-provided GPU resources. We will use a consistent GPU model for the main comparisons and may evaluate on another GPU architecture if resources permit.

For background, we will consult the MoE model literature and GPU programming documentation, including Shazeer et al., ``Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,'' ICLR 2017, and NVIDIA's *CUDA C++ Programming Guide*. We may consult additional relevant implementations and papers as the design evolves.

Goals and Deliverables

Plan to achieve. We will implement and compare a sequence of correct versions under a common benchmark:

- **Baseline:** process each active expert separately using a straightforward matrix-multiplication implementation.
- **Grouped execution:** organize expert computations to reduce per-expert scheduling or launch overhead where practical.
- **Tile-based scheduling:** partition expert computations into GPU work tiles and schedule them across experts to expose more parallelism and improve balance.

We will check outputs against a reference with an appropriate numerical tolerance, then compare latency and speedup across token counts, expert counts, and routing distributions, including balanced and skewed cases. We will make measurement scope explicit, including whether routing, data movement, and launch overhead are timed, and use the

same scope for comparisons. Profiling and controlled experiments will help explain changes in performance, including cases where an optimization does not help. Our deliverables will include the implementations, benchmark methodology, results, and final report and poster.

We aim to demonstrate a measurable improvement over the baseline on at least some representative imbalanced workloads; the amount of speedup will depend on dimensions and hardware, so we do not assume a fixed target before measurement. The progression is motivated by reducing overhead from small expert workloads and increasing the pool of independently schedulable tiles.

Hope to achieve. If time permits, we will investigate dynamic scheduling, such as a shared work queue, and compare its balancing benefits with its coordination overhead. We may also evaluate on an additional GPU and examine portability across workload sizes.

If progress is slower than expected. We will prioritize a correct baseline, at least one optimized execution strategy, and a careful comparison across balanced and imbalanced inputs; the second optimization can be narrowed to a smaller set of workloads.

Platform Choice

CUDA on an NVIDIA GPU is a natural fit because expert matrix operations offer substantial parallelism, while GPU thread-block scheduling and memory hierarchy make the effects of workload imbalance observable. A cloud GPU instance provides access to a CUDA development environment and a consistent platform for the main experiments. GPU availability, memory capacity, cost, and profiling permissions will guide our platform selection and workload sizes. An additional GPU, if available, would allow us to investigate how conclusions change across platforms.

Schedule

- **Nov. 9–15:** Set up the benchmark and reference, define representative routing distributions, and implement and validate the per-expert baseline.
- **Nov. 16–22:** Profile the baseline, develop grouped execution, and begin controlled comparisons and tile-based design.
- **Nov. 23–29:** Submit the milestone report on Nov. 23; implement and validate tile-based scheduling, allowing for reduced availability during Thanksgiving week.
- **Nov. 30–Dec. 6:** Finish core experiments, investigate bottlenecks and parameter sensitivity, and explore stretch goals if time permits.
- **Dec. 7–11:** Consolidate results, finalize the report by Dec. 10, and prepare the poster for the Dec. 11 session.